PÓS TECH FIAP · TECH CHALLENGE · FASE 4

ToggleMaster

Relatório de entrega: observabilidade, resposta a incidentes e self-healing no EKS

PARTICIPANTES

Gabriel Pinelli Silva	RM373763	@Tocaccelli
João Vitor de Jesus Ciardullo	RM372155	@joaociardullo
Douglas Deveza dos Santos	RM373827	@Douglasdeveza
João Carlos da Silva Brito	RM371738	@Durmiand
João Gabriel da Cruz Sales	RM372444	@jgabrieldev1

LINKS DA ENTREGA

REPOSITÓRIO
github.com/fiap-devops-arqcloud-2026/tech-challenge-04

VÍDEO DE DEMONSTRAÇÃO
a incluir link do vídeo gravado em 2026-11-11

SELF-HEALING EXECUTADO NA GRAVAÇÃO
execução 42956432177 · GitHub Actions

5
microsserviços instrumentados com OpenTelemetry

6
Applications Synced/Healthy no ArgoCD

2 min 50 s
da falha proposital à volta do HTTP 200

0
ações humanas na recuperação

Edição 2026-11-11 · prints do ensaio integrado de 2026-11-09 · execução gravada em 2026-11-11

SUMÁRIO DA ENTREGA

O que este relatório comprova

Cada exigência do relatório (enunciado, p. 6) e onde encontrá-la. A matriz da p. 15 cobre também os requisitos técnicos e o vídeo.

O desafio

O evaluation-service falhou em silêncio: o time levou 6 horas para ser avisado e mais 4 para achar a causa. O objetivo da Fase 4 é observabilidade total e resposta ativa: trace para lentidão, logs centralizados para erros, plantão acionado e uma tentativa automática de cura.

O que construímos

Sobre a base das fases 1 a 3 (cinco microsserviços, Terraform, CI/CD com DevSecOps e EKS com ArgoCD), a Fase 4 entrega Prometheus, Loki e Grafana por GitOps, um OpenTelemetry Collector central, APM no New Relic e uma cadeia de resposta: alerta no Grafana, incidente no PagerDuty, aviso no Slack e self-healing no GitHub Actions.

A prova

Na gravação de 2026-11-11, uma falha proposital no flag-service foi detectada, comunicada e corrigida sem intervenção humana em 2 min 50 s (p. 5). Os prints vêm do ensaio integrado de 2026-11-09, com a mesma implantação e a mesma configuração.

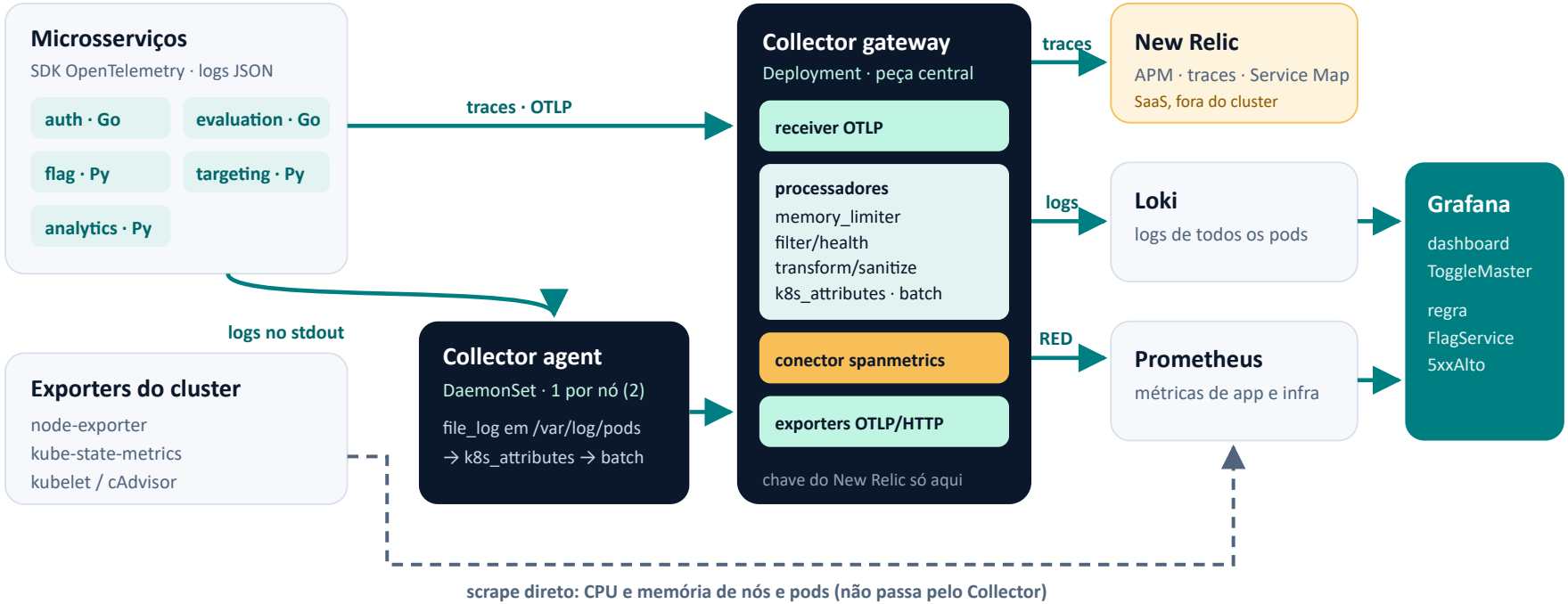
EXIGÊNCIA DO RELATÓRIO	ONDE
✓ Nomes, RMs e usernames	p. 1
✓ Links do repositório e do vídeo	p. 1
✓ Print do dashboard do Grafana	p. 6
✓ Print de um trace distribuído no APM	p. 8
✓ Print da notificação de incidente no ChatOps	p. 10
✓ Print do log e da execução do self-healing	p. 11
✓ Justificativa: arquitetura do OTel	p. 3
✓ Justificativa: Datadog × New Relic e OpsGenie × PagerDuty	p. 4

Horários em America/Sao_Paulo (UTC-03), salvo quando indicado UTC. Os prints não foram editados: só recortados para ampliar a região que comprova cada item.

Exigência p. 4 e 6

Arquitetura do OpenTelemetry

O Collector é a peça central: recebe, processa e roteia traces, logs e métricas de aplicação para o backend certo. As aplicações não conhecem os destinos.



Agent em cada nó

DaemonSet com um pod por nó. Lê os arquivos de log de todos os contêineres em `/var/log/pods`, acrescenta namespace, pod e deployment, e envia ao gateway por OTLP/gRPC. Os logs entram no Loki sem mudar o código.

Gateway central

Recebe os traces dos SDKs, descarta spans de `/health` e mantém só atributos permitidos antes de sair do cluster. Exporta traces ao New Relic e logs ao Loki; o **spanmetrics** gera taxa, erros e latência (RED) para o Prometheus.

Por que este desenho

- Trocar de APM é trocar um exporter, não o código dos cinco serviços.
- A chave do New Relic fica só no gateway.
- CPU e memória já vêm do Prometheus por scrape; o Collector não duplica esse caminho.

Exigência p. 6

Por que New Relic e PagerDuty

Os critérios foram custo sem prazo, encaixe com o OpenTelemetry já implantado e disponibilidade do serviço.

APM

New Relic em vez de Datadog

CRITÉRIO	NEW RELIC	DATADOG
Plano gratuito	Permanente: 100 GB/mês e 1 usuário completo, com APM, tracing e Service Map	Gratuito só para infraestrutura; APM no teste de 14 dias ou em plano pago
Entrada OpenTelemetry	Endpoint OTLP nativo: o gateway exporta direto, sem agente proprietário	OTLP pelo Datadog Agent ou por exporter próprio
No desafio	Trace distribuído e mapa de dependências a partir dos spans OTel	Recursos equivalentes, com custo após o teste

Decisão: New Relic. Custo zero sem prazo de expiração e ingestão OTLP direta do gateway: o mesmo dado que alimenta o Prometheus vira trace e Service Map no APM.

GESTÃO DE INCIDENTES

PagerDuty em vez de OpsGenie

CRITÉRIO	PAGERDUTY	OPSGENIE
Disponibilidade	Ativo; conta criada para o projeto	Venda encerrada em 2025-06-04, sem contas novas; fim do suporte em 2027-04-05
Integração com o Grafana	Contact point nativo (Events API v2): abre e resolve o incidente sozinho	Também tinha, mas sem conta nova não há como usar
ChatOps	App do PagerDuty no Slack publica o cartão do incidente	Migração indicada para o Jira Service Management

Decisão: PagerDuty. O OpsGenie não aceita contas novas desde 2025-06-04. O PagerDuty integra nativamente com o Grafana e o Slack, e o incidente fecha sozinho quando o alerta se resolve.

Fontes, consultadas em 2026-11-11: newrelic.com/pricing; datadoghq.com/pricing; atlassian.com/licensing/opsgenie; documentação do Grafana sobre o contact point do PagerDuty. Links na p. 17.

A PROVA REAL · GRAVAÇÃO DE 2026-11-11

Da falha à recuperação, sem intervenção humana

Vídeo: incidente e self-healing

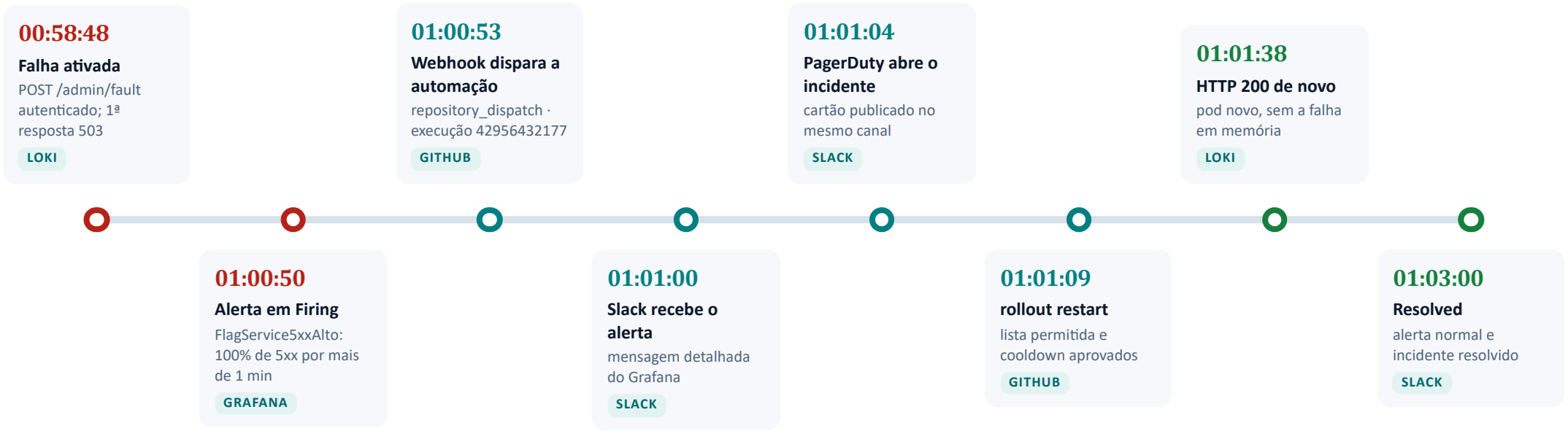
Falha proposital no flag-service, medida nos próprios registros: logs no Loki, log do GitHub Actions e mensagens do Slack.

2 min 02 s
da falha ao Firing (regra: acima de 10% por 1 min)

19 s
do Firing ao rollout restart

2 min 50 s
da falha à volta do HTTP 200

159
respostas 503 vistas pelo gerador de tráfego



Falha 503 nas rotas de flags → **Métrica** spanmetrics no Prometheus → **Alerta** Grafana em Firing → **Incidente** PagerDuty → **Comunicação** Slack → **Correção** GitHub Actions → **Recuperação** HTTP 200

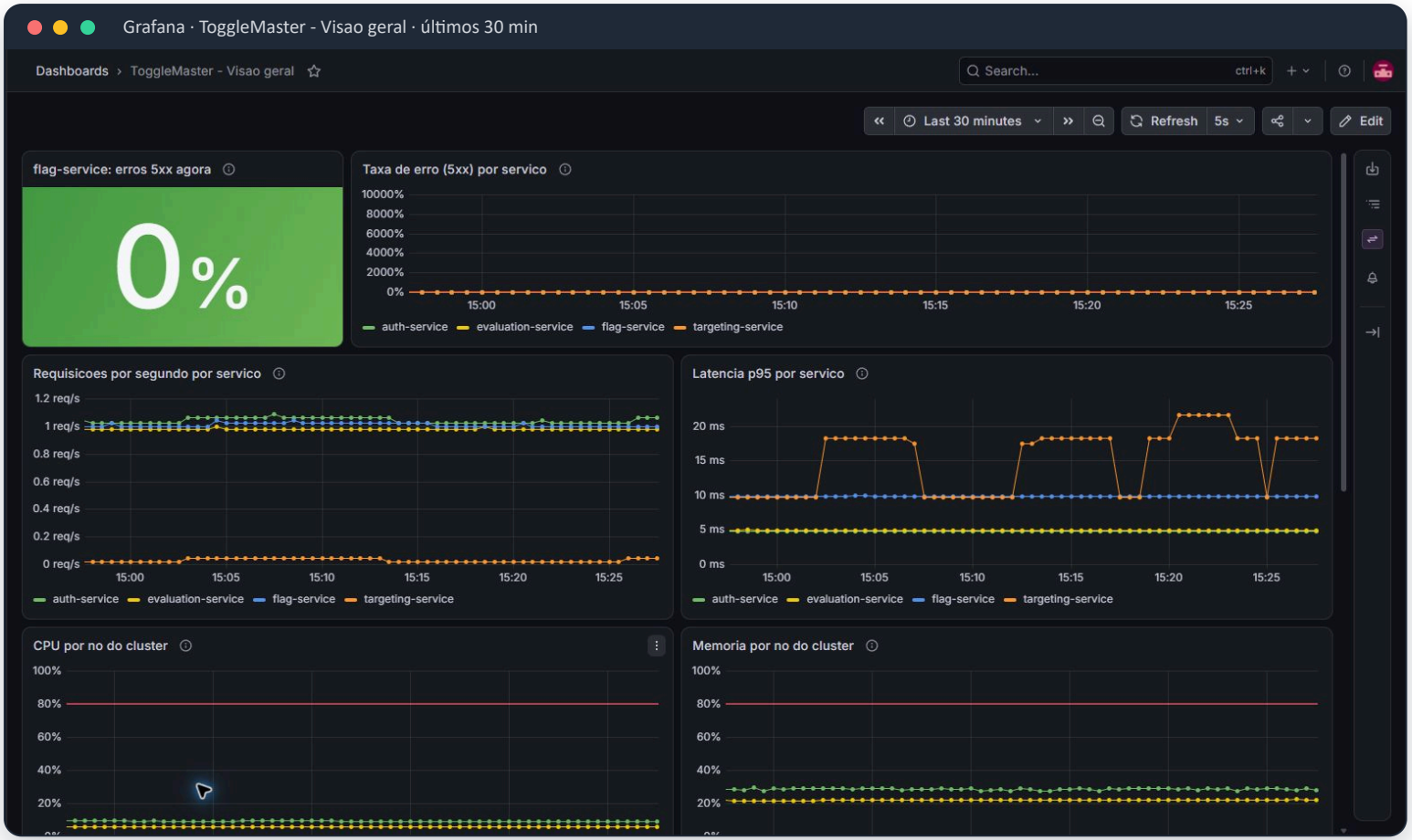
O ensaio de 2026-11-09 percorreu a mesma cadeia: Firing às 15:59:20, restart às 15:59:39, HTTP 200 às 16:00:19 e Resolved às 16:01:30 (execução 42679005683, incidente #4 no PagerDuty). O /health seguiu saudável durante a falha: o Kubernetes sozinho não teria reiniciado o pod.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 1 DE 4

Dashboard do Grafana: tráfego, erros e latência

Print exigido

Dashboard customizado ToggleMaster - Visao geral, versionado no GitOps.



2026-11-09 · ensaio

Grafana 13.3

dados reais

O QUE O PRINT MOSTRA

- **Erros 5xx agora** do flag-service: 0% antes da falha.
- **Taxa de erro (5xx)** e **requisições por segundo** de cada serviço HTTP.
- **Latência p95**: 95% das requisições abaixo de 25 ms.
- **CPU e memória** de cada nó do EKS.

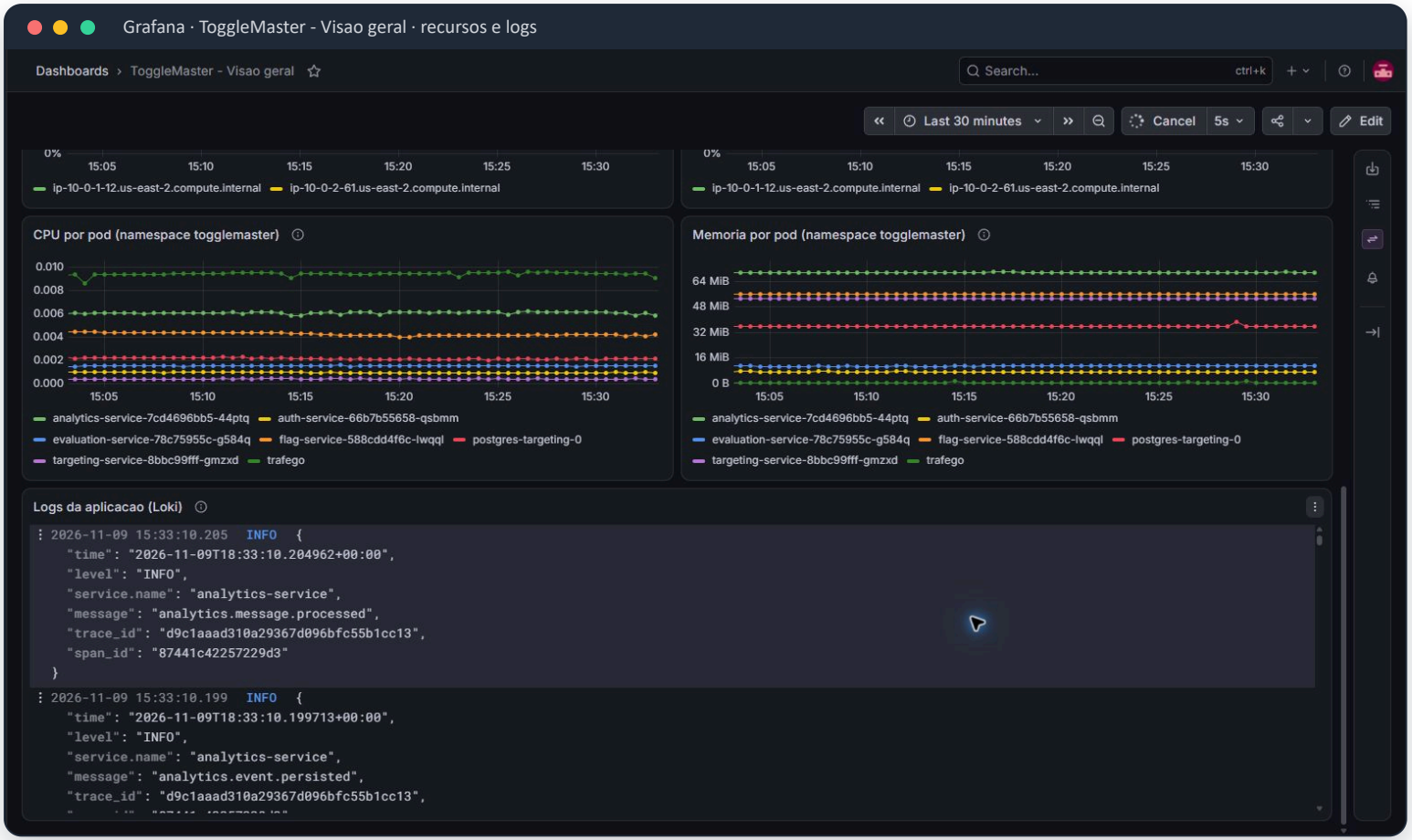
As métricas de requisição vêm dos traces (spanmetrics); as de nó, do Prometheus por scrape. O eixo do painel de erros foi limitado a 100% no repositório depois desta captura.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 1 DE 4

Dashboard do Grafana: recursos e logs em tempo real

Print exigido

Continuação do mesmo dashboard: consumo por pod e o painel de logs do Loki.



2026-11-09 · ensaio · Loki · JSON com trace_id

O QUE O PRINT MOSTRA

- **CPU e memória por pod** do namespace togglemaster.
- **Logs da aplicação**, consultados no Loki em tempo real.
- Cada log traz `service.name` e `trace_id`: o evento leva direto ao trace no APM.

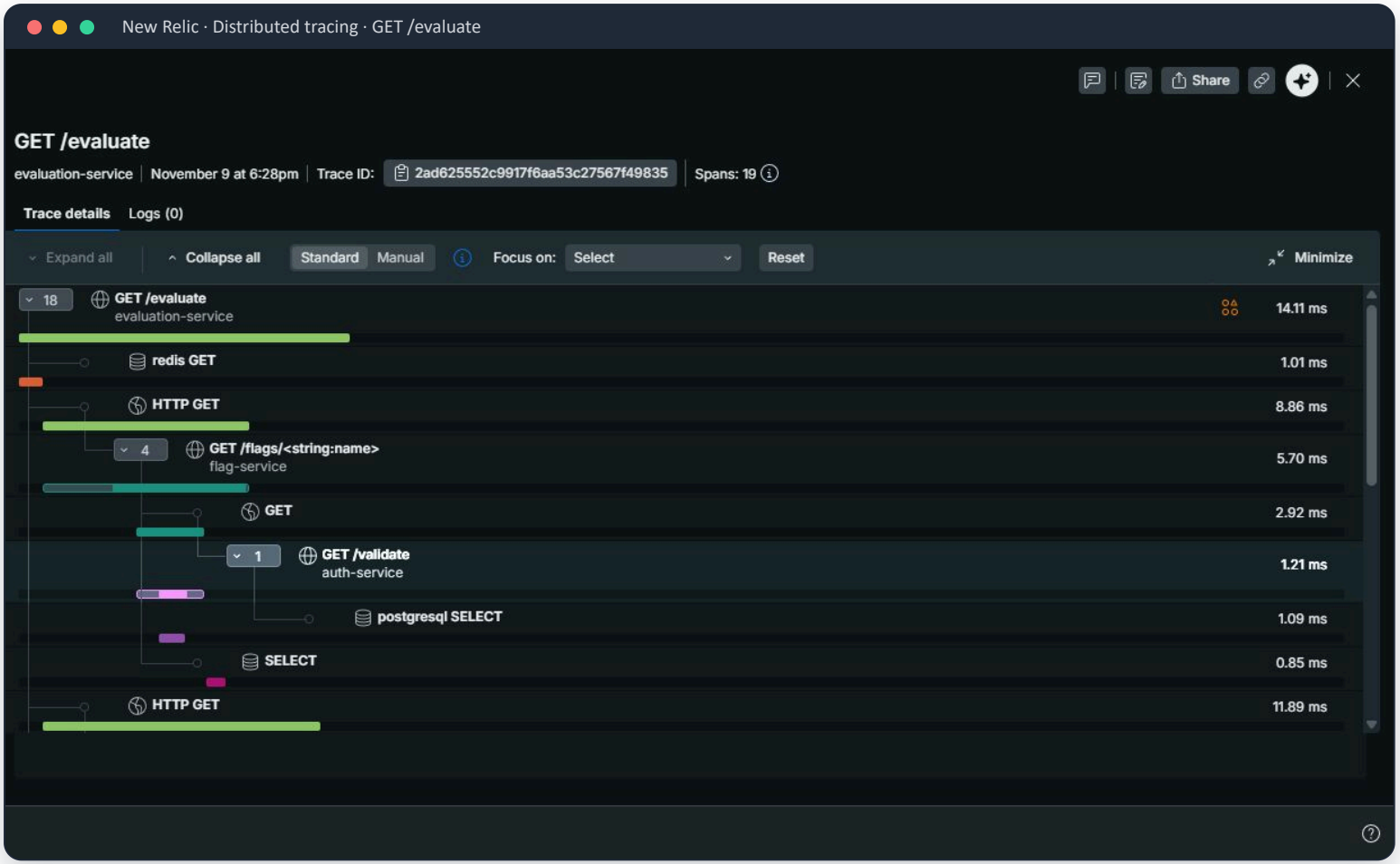
Os logs chegam pelos agents do Collector, sem alteração no código de log das aplicações.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 2 DE 4

Trace distribuído no New Relic

Print exigido

Uma requisição ao evaluation-service e o caminho que ela percorre pelos serviços e bancos.



2026-11-09 18:28 UTC · 19 spans · 0 erros · 56,63 ms

O QUE O PRINT MOSTRA

- Entrada em **GET /evaluate** no evaluation-service.
- Consulta ao **Redis** (1,01 ms) e chamada ao flag-service **GET /flags/<name>**.
- O flag-service valida a chave no auth-service (**GET /validate**), que consulta o **PostgreSQL**.
- Duração de cada span na direita: localiza a operação mais lenta.

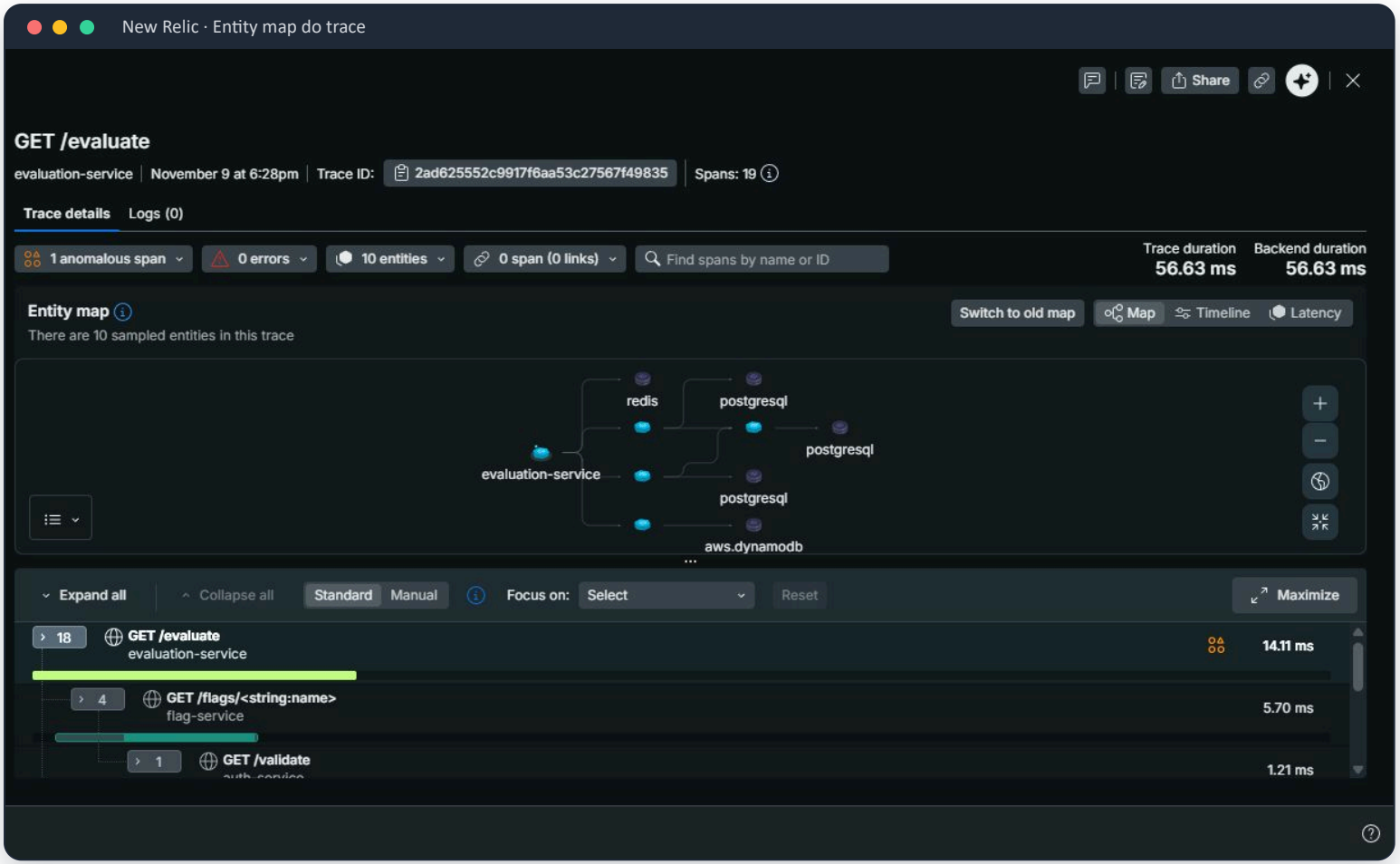
Trace 2ad625552c9917f6aa53c27567f49835, com as operações expandidas.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 2 DE 4

O mesmo trace no mapa de entidades

Print exigido

Dez entidades na mesma execução: os serviços e as dependências de dados.



10 entidades

SQS → analytics

DynamoDB

O QUE O PRINT MOSTRA

- evaluation-service no centro, com Redis e três bancos PostgreSQL.
- O contexto atravessa a fila SQS: o analytics-service grava no **DynamoDB** dentro do mesmo trace.
- Fluxo assíncrono: a resposta HTTP não espera a persistência analítica.

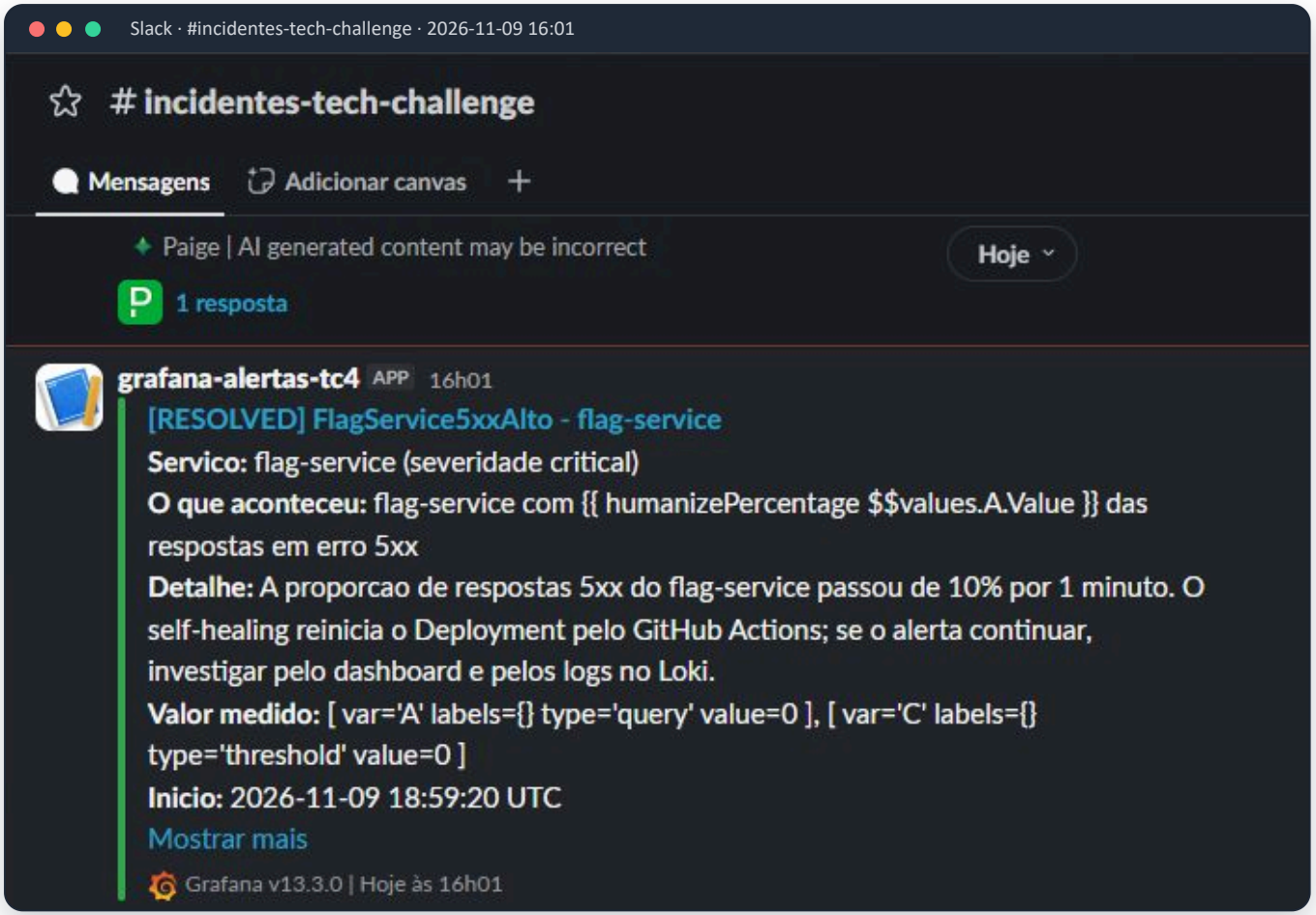
O Service Map global com os cinco microsserviços é mostrado no vídeo, no bloco de APM.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 3 DE 4

Notificação do incidente no ChatOps

Print exigido

Canal #incidentes-tech-challenge no Slack: o Grafana publica o alerta detalhado e o PagerDuty, o cartão do incidente.



NA GRAVAÇÃO DE 2026-11-11 · MENSAGENS LIDAS PELA API DO SLACK

grafana-alertas-tc4 01:01
[FIRING] FlagService5xxAlto - flag-service
Serviço: flag-service (critical) · Valor medido: A = 1 (100%)
Início: 04:00:50 UTC · Ação automática: self-healing via GitHub Actions

PagerDuty 01:01
[FIRING] FlagService5xxAlto - flag-service
Urgency: High · Service: ToggleMaster Fase 4 · Triggered via: Grafana

grafana-alertas-tc4 01:03
[RESOLVED] FlagService5xxAlto - flag-service
Valor medido: A = 0 · regra, runbook e link para silenciar

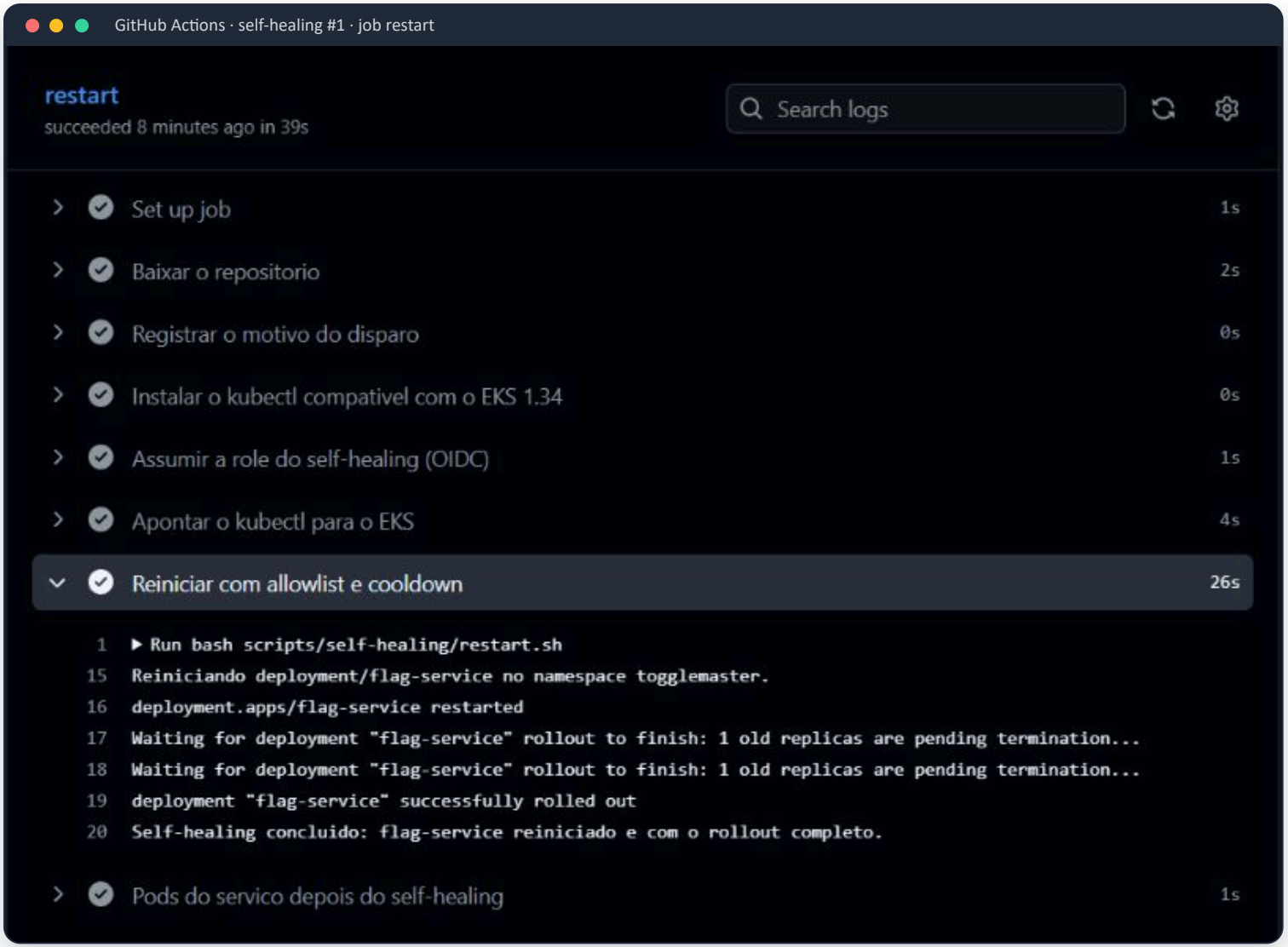
A linha "O que aconteceu" mostrou o texto do modelo no lugar do percentual; o valor aparece em "Valor medido". O modelo foi corrigido no repositório.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 4 DE 4

Log da automação de self-healing

Print exigido

GitHub Actions disparado pelo alerta via repository_dispatch; o passo de restart registra a ação corretiva.



2026-11-09 · ensaio · sucesso

passo em 26 s

O QUE O PRINT MOSTRA

- **Assumir a role do self-healing (OIDC)**: credencial temporária, sem chave guardada.
- **Reiniciar com allowlist e cooldown**: confere a lista permitida e 10 min desde o último restart.
- `deployment.apps/flag-service restarted` e `successfully rolled out`.
- **Pods do serviço depois do self-healing**: o pod novo em execução.

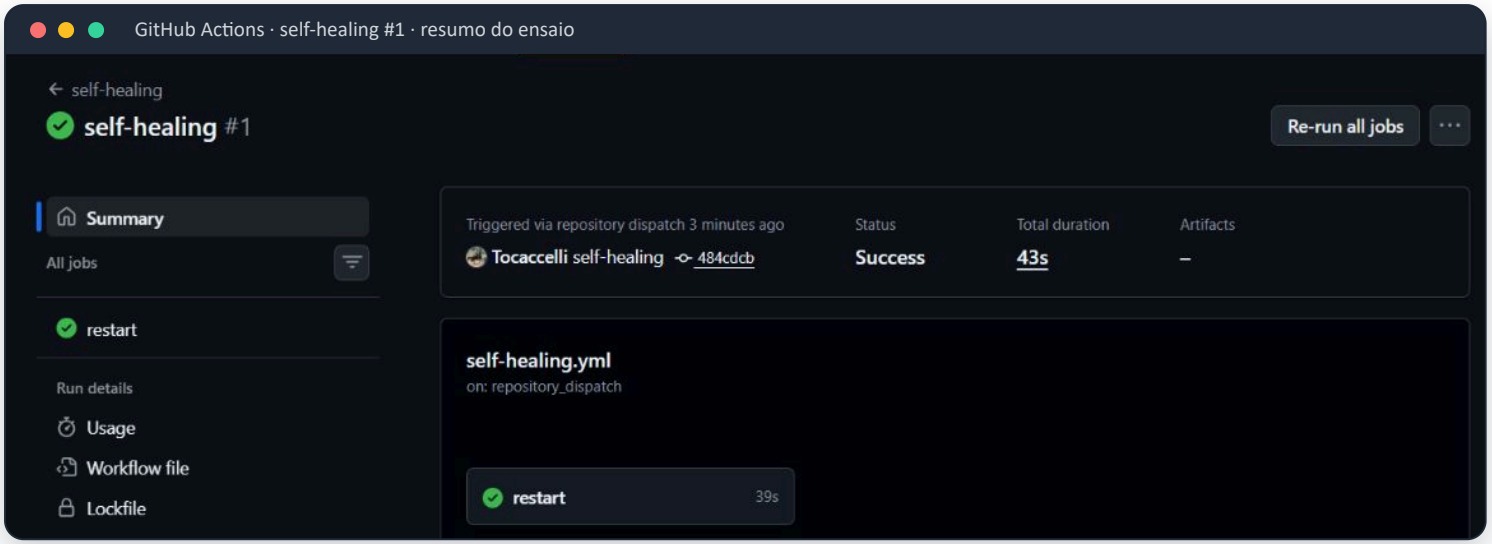
Execução 42679005683, iniciada por repository_dispatch, sem nenhum clique humano.

EVIDÊNCIA VISUAL OBRIGATÓRIA · 4 DE 4

Print exigido

Execução do self-healing: ensaio e gravação

O resumo mostra o gatilho automático; o trecho ao lado é o log da execução feita durante a gravação do vídeo.



Triggered via repository dispatch

Success · 43 s

O QUE O PRINT MOSTRA

- O run nasce do webhook do Grafana: **Triggered via repository dispatch**.
- Job **restart** concluído em 39 s; execução total em 43 s.

GRAVAÇÃO DE 2026-11-11 · EXECUÇÃO 42956432177 · HORÁRIOS UTC

```
04:01:07 Run bash scripts/self-healing/restart.sh
04:01:07   SERVICE: flag-service
04:01:07   ALERT_NAME: FlagService5xxAlto
04:01:07   ALERT_STATUS: firing
04:01:07   COOLDOWN_SECONDS: 600
04:01:08 Reiniciando deployment/flag-service no namespace togglemaster.
04:01:09 deployment.apps/flag-service restarted
04:01:10 Waiting for deployment "flag-service" rollout to finish:
                1 old replicas are pending termination...
04:01:31 deployment "flag-service" successfully rolled out
04:01:31 Self-healing concluido: flag-service reiniciado e com o rollout completo.
04:01:32 NAME                            READY   STATUS    AGE
04:01:32 flag-service-cb85cd74d-lz4wq    1/1     Running   23s
```

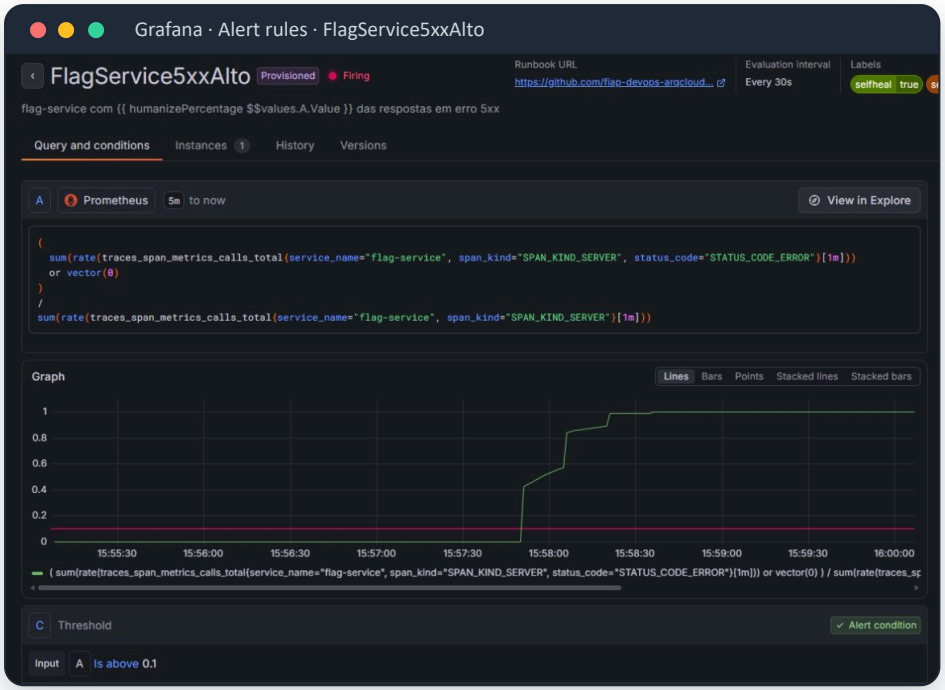
Trecho exportado com `gh run view 42956432177 --log`. As credenciais temporárias aparecem mascaradas no log original e foram omitidas aqui.

EVIDÊNCIA DE APOIO

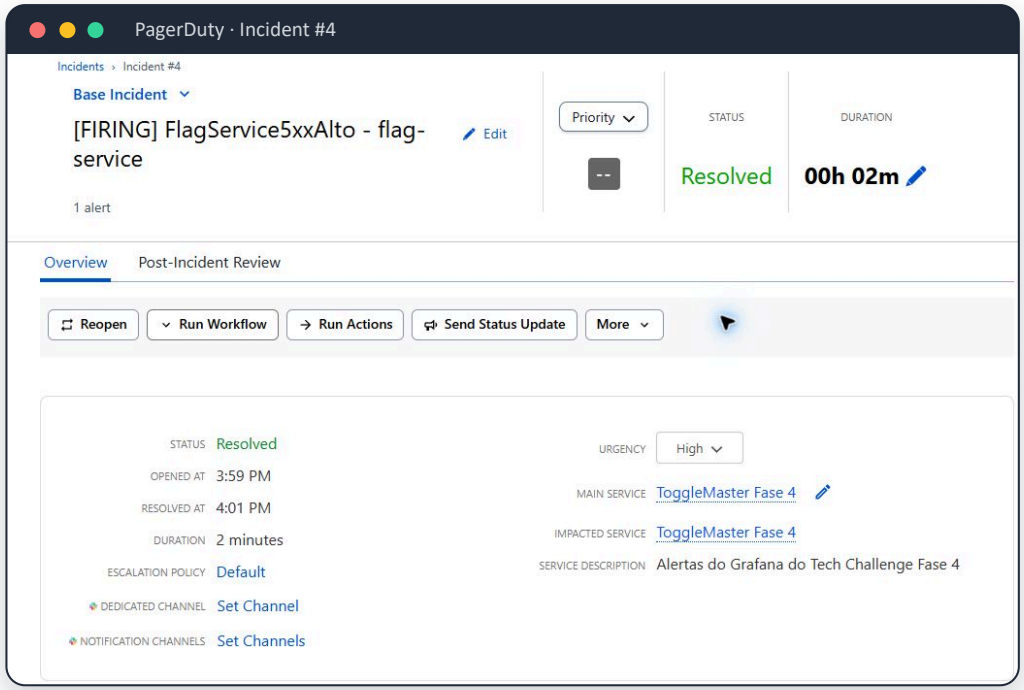
Apoio

Alerta em Firing e incidente no PagerDuty

Os dois passos da prova real que vêm antes do ChatOps: a regra disparando e o incidente aberto sem clique humano.



Firing às 15:59:20: a razão 5xx ÷ total do flag-service sobe de 0 para 1 e cruza o limite de 0,1 (linha vermelha). Avaliação a cada 30 s, período pendente de 1 min. Rótulos selfheal=true, service=flag-service e severity=critical.



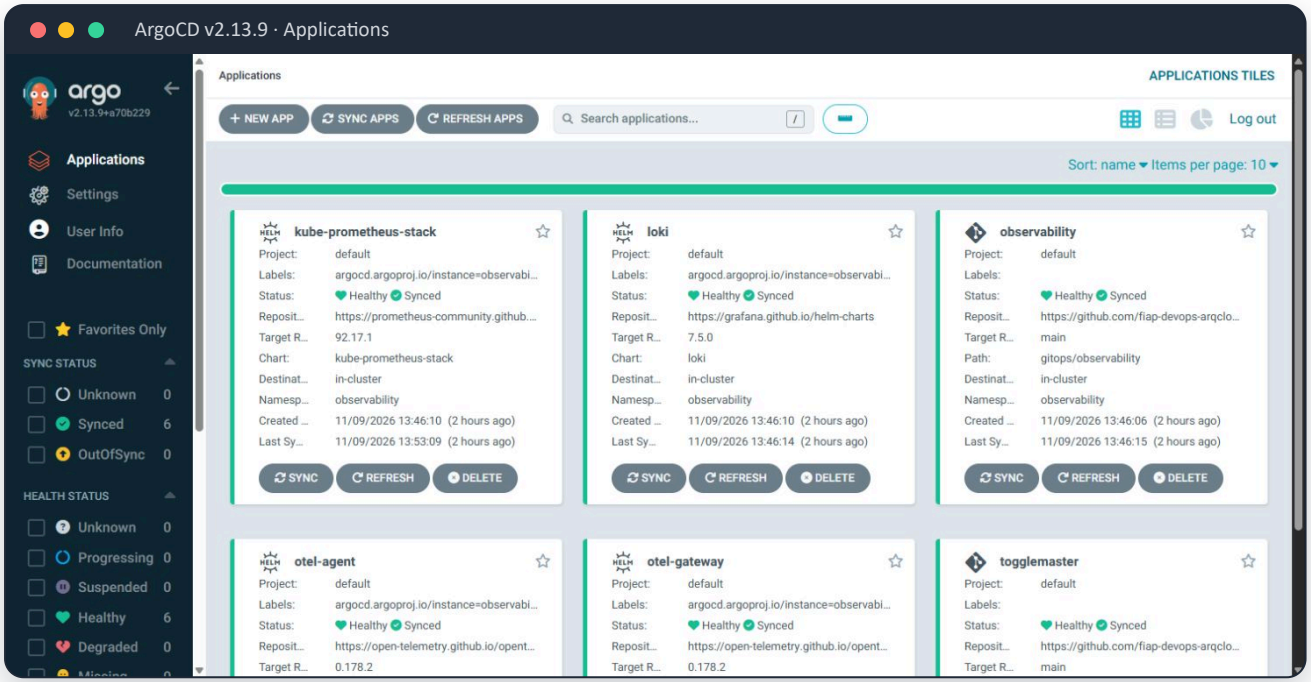
Incidente #4: aberto às 3:59 PM, resolvido às 4:01 PM, duração de 2 minutos, urgência High, serviço ToggleMaster Fase 4.

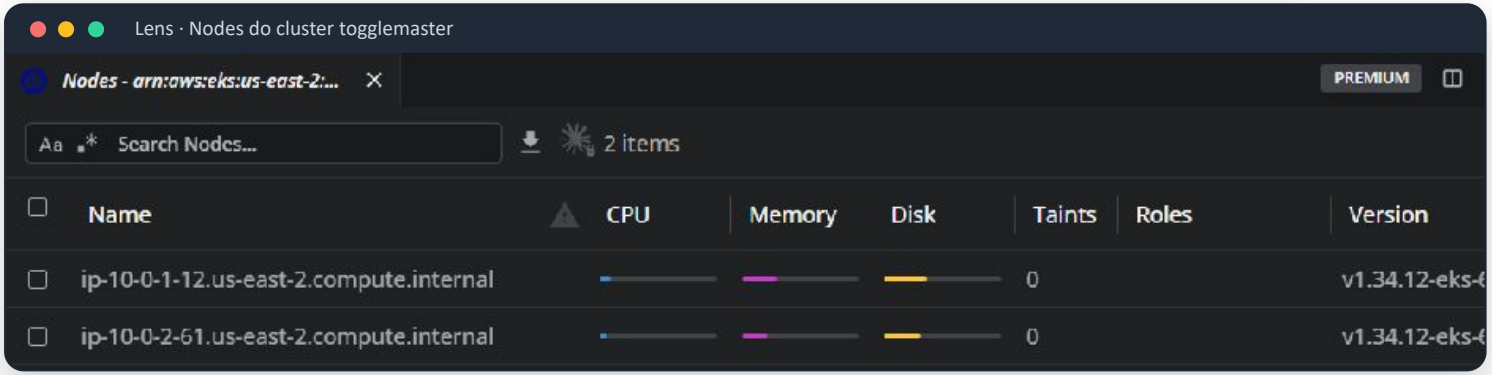
EVIDÊNCIA DE APOIO

Cluster EKS e GitOps em operação

Apoio

A base das fases anteriores continua rodando com a nova stack: o ArgoCD reconcilia aplicação e observabilidade.





6 Synced · 6 Healthy · 2 nós Ready

O QUE OS PRINTS MOSTRAM

- **togglemaster**: os cinco microsserviços, pelo overlay de produção.
- **observability**, que organiza kube-prometheus-stack, Loki, otel-gateway e otel-agent como charts Helm.
- Abaixo, os dois nós m7i-flex.large do EKS, com Kubernetes v1.34.

COBERTURA DOS REQUISITOS · 1 DE 2

Enunciado p. 2–6

Matriz: requisito, solução e onde comprovar

A regra de avaliação (p. 5) desconta o que não for demonstrado no vídeo ou documentado no relatório. Esta tabela aponta a prova de cada item.

#	REQUISITO	COMO FOI ATENDIDO	ONDE COMPROVAR
BASE DAS FASES 1 A 3 · P. 2			
1	**5 microsserviços conteinerizados**	auth e evaluation em Go; flag, targeting e analytics em Python; imagens no ECR	Vídeo: visão geral · services/
2	**Infraestrutura por Terraform**	3 camadas com estados próprios: base, cluster (EKS, RDS, Redis, IRSA) e k8s (ArgoCD, Secrets)	Vídeo: visão geral · terraform/
3	**CI/CD com DevSecOps**	build, testes, lint, SAST (gosec, bandit), SCA e scan de imagem (Trivy); publicação no ECR por OIDC	Vídeo: visão geral · .github/workflows/
4	**Kubernetes gerenciado por GitOps**	EKS 1.34 e ArgoCD v2.13.9 com 6 Applications Synced/Healthy	Vídeo: visão geral · p. 14 · gitops/
MONITORAMENTO OPEN SOURCE · P. 3			
5	**Prometheus por Helm no GitOps**	kube-prometheus-stack como Application própria	gitops/observability/applications
6	**Loki com os logs dos contêineres**	Loki por Helm; agents em DaemonSet leem /var/log/pods de cada nó	Vídeo: métricas e logs · p. 7
7	**Dashboard customizado no Grafana**	recursos do cluster, requisições por serviço, erros, p95 e logs em tempo real	Vídeo · p. 6–7 · togglemaster-overview.json
OPENTELEMETRY · P. 4			
8	**OTel Collector como peça central**	gateway e agents; traces → New Relic, logs → Loki, spanmetrics → Prometheus	Vídeo · p. 3 · otel-gateway.yaml, otel-agent.yaml
APM E TRACING · P. 4			
9	**Ferramenta APM comercial**	New Relic, com ingestão OTLP a partir do gateway	p. 4
10	**Instrumentação com OpenTelemetry**	SDK OTel em Go e Python (telemetry.go, telemetry.py); logs JSON com trace_id	services/*/telemetry.*
11	**Distributed tracing do evaluation-service**	trace de GET /evaluate com 19 spans e o caminho entre os serviços	Vídeo: APM · p. 8
12	**Service Map dos 5 microsserviços**	mapa de dependências no New Relic	Vídeo: APM · p. 9

Enunciado p. 4–6

Matriz: resposta a incidentes e entregáveis

Continuação da matriz. A coluna da direita indica o bloco do vídeo, a página deste relatório ou o caminho no repositório.

#	REQUISITO	COMO FOI ATENDIDO	ONDE COMPROVAR
ALERTAS, INCIDENTES E SELF-HEALING · P. 4–5			
13	**Alerta inteligente**	FlagService5xxAlto: 5xx do flag-service acima de 10% por 1 min, avaliado a cada 30 s	Vídeo · p. 13 · rules.yaml
14	**Incidente aberto automaticamente**	PagerDuty pela Events API v2; abre e resolve sozinho	Vídeo · p. 13, 5 · contactpoints.yaml
15	**Notificação detalhada no ChatOps**	Slack: alerta detalhado do Grafana e cartão do PagerDuty	Vídeo · p. 10
16	**Self-healing sem intervenção humana**	repository_dispatch → OIDC → lista permitida e cooldown de 600 s → rollout restart	Vídeo · p. 11–12 · self-healing.yml, restart.sh
ENTREGÁVEIS · P. 5–6			
17	**Código IaC/GitOps com a stack e o Collector**	gitops/observability e terraform/k8s	repositório
18	**Código das aplicações instrumentado**	cinco serviços em services/	repositório
19	**Scripts do self-healing**	scripts/self-healing, workflow self-healing e RBAC em gitops/base	repositório
20	**Vídeo de até 25 min**	visão geral, métricas e logs, APM e prova real, gravados em 2026-11-11	link na capa
21	**Relatório em PDF**	identificação, links, 4 prints e justificativa técnica	este documento, p. 1–12

ENCERRAMENTO

Limites conhecidos, reprodução e referências

O que a solução não faz, como reproduzir o ambiente e de onde vieram os dados deste relatório.

Limites conhecidos

- Reiniciar resolve falhas transitórias do processo, não uma dependência fora do ar. Com o banco parado, o alerta continuaria e o PagerDuty acionaria o plantão.
- O cooldown de 600 s evita reinícios em laço; um segundo alerta nesse intervalo fica só com o atendimento humano.
- O analytics consome a fila SQS de forma assíncrona; por isso não aparece com tráfego HTTP no dashboard.

Como reproduzir

- Terraform em três camadas: base, cluster e k8s, cada uma com plano salvo.
- O ArgoCD sincroniza gitops/overlays/prod e gitops/observability a partir da main.
- Passo a passo em docs/GUIA_DE_REPRODUCAO.md; requisitos em docs/requisitos/README.md.
- Prints e índice em docs/evidencias/; roteiro do vídeo em docs/gravacao/.

Execuções citadas

- Gravação: self-healing 42956432177
- Ensaio: self-healing 42679005683
- Repositório: github.com/fiap-devops-arqcloud-2026/tech-challenge-04

Referências

- FIAP. Tech Challenge Fase 4, enunciado (PDF no repositório), p. 2–6.
- New Relic. Pricing: newrelic.com/pricing.
- Datadog. Pricing: datadoghq.com/pricing.
- Atlassian. Opsgenie end of sale e end of support: atlassian.com/licensing/opsgenie.
- Grafana. Configure PagerDuty for alerting: grafana.com/docs/…/pager-duty.
- OpenTelemetry. Collector Helm chart, presets de coleta de logs: opentelemetry.io/docs/…/collector.

Sites consultados em 2026-11-11. Dados da gravação obtidos do Loki, do log do GitHub Actions e da API do Slack.